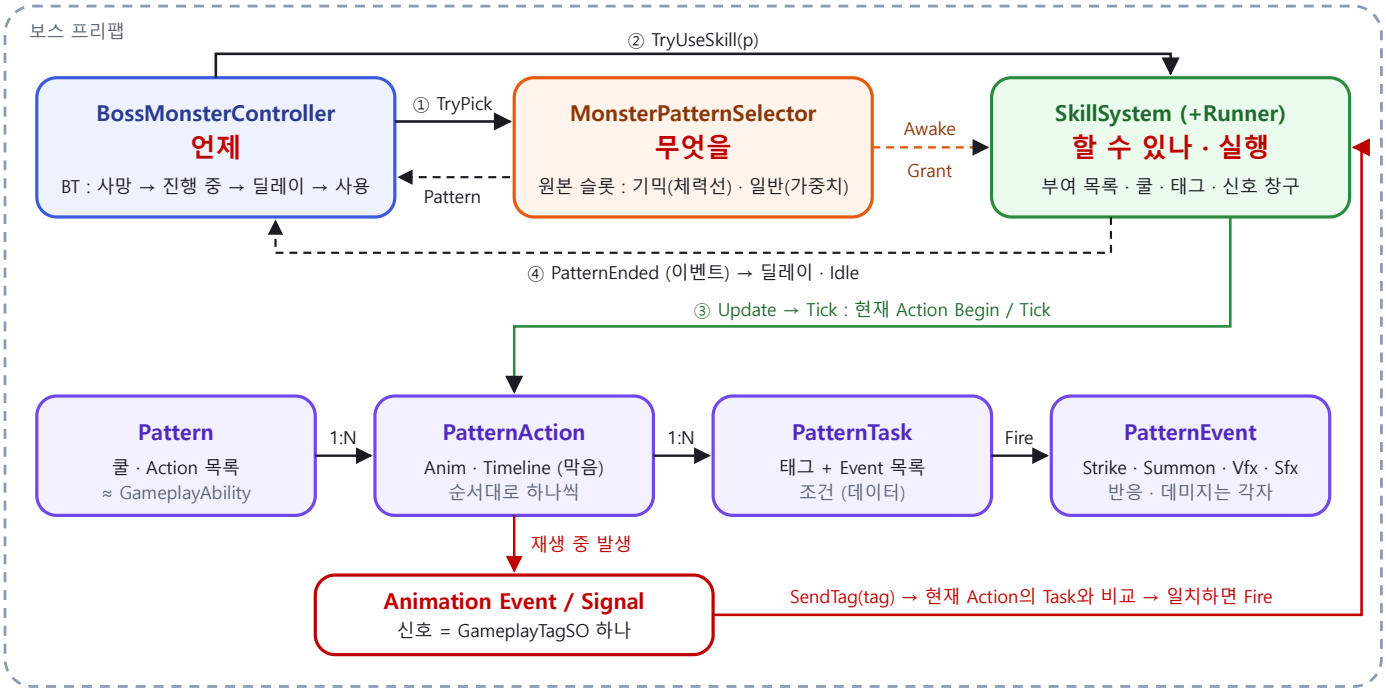
COMBATABILITY · 03 · 2026-10-27

SkillSystem 구조 점검 — 지금 구조 · 평가 · 할 일 · GAS 비교

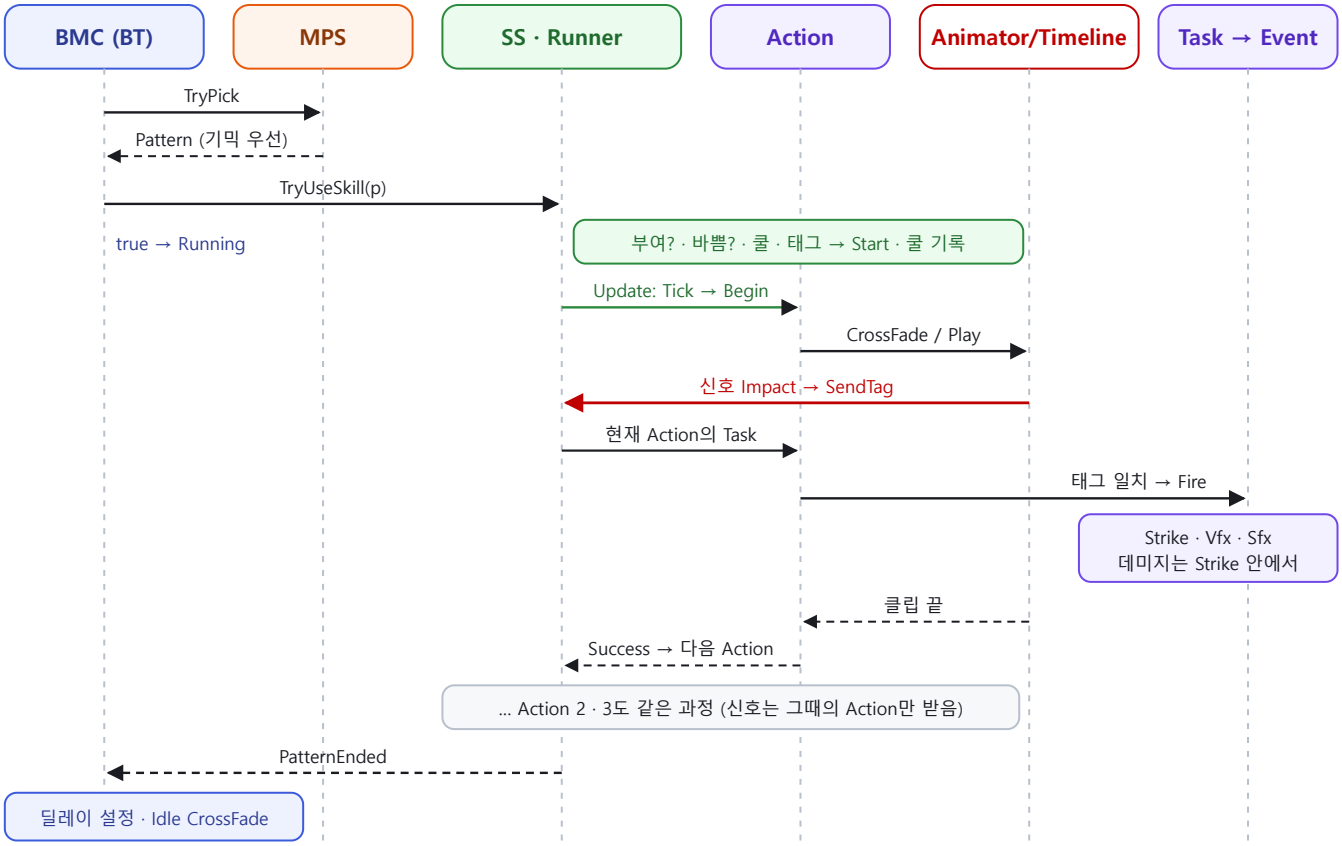
보스 프리팹 하나에 **네 개의 역할**이 나뉘어 있다. **언제**(BMC) · **무엇을**(MPS) · **할 수 있나 + 실행**(SS) · **어떻게**(Pattern 데이터). 패턴 안에서는 애니메이션/타임라인이 **신호(태그)**만 보내고, **반응(Event)**은 데이터로 붙인다.

1. 구조도

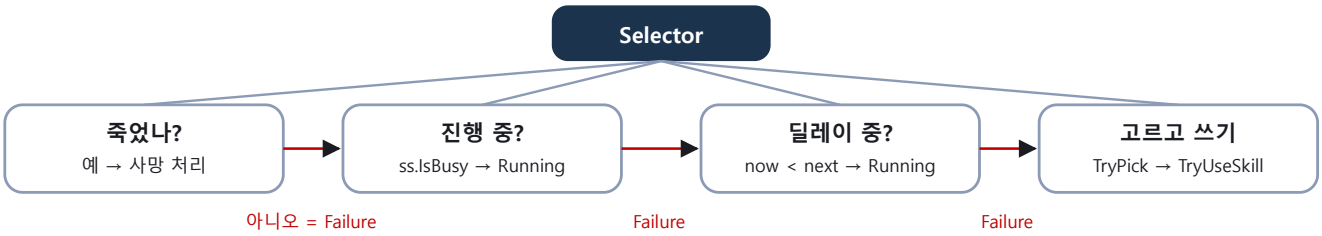


부품	책임	아는 것 / 모르는 것
BossMonsterController	언제 쓸지	BT 타이밍만 안다. 어떤 패턴이 있는지는 모른다.
MonsterPatternSelector	무엇을 쓸지	슬롯의 원본(가중치·체력선). Awake에 SS로 Grant. 실행은 모른다.
SkillSystem	쓸 수 있나 + 실행	부여 목록·쿨·태그·Runner. **왜** 이 패턴이 골라졌는지는 모른다. 플레이어와 공용.
Pattern → Action → Task → Event	어떻게	전부 프리팹 위 데이터. 애니메이션은 태그만 보내고, 반응은 Task에 적힌 Event가 정한다.

2. 한 번의 패턴이 도는 순서

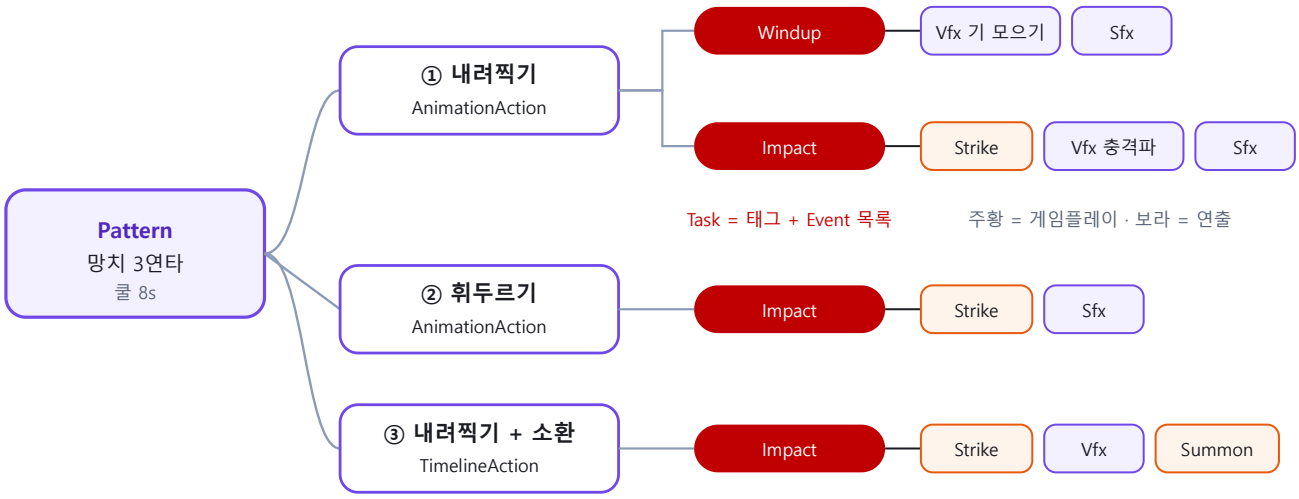


3. BT — Failure일 때만 다음 칸으로



Running 또는 Success가 나오면 그 프레임은 거기서 끝. "끝났다"는 상태가 아니라 **PatternEnded 이벤트**로만 전달.

4. 예시 — 망치 3연타 한 패턴이 데이터로 어떻게 생겼나



같은 Impact 태그, 다른 반응 — 애니메이션은 "찍었다"만 알리고, 무엇을 할지는 Action마다 붙은 데이터가 정한다.

신호와 반응을 나눈 덕분에 생기는 것

1. 같은 클립, 다른 보스

- 클립에는 태그만 박혀 있다. 두 번째 보스는 Task의 Event만 바꿔 같은 클립을 쓴다 (코드 0줄).

2. 구독 · 해제가 없다

- 신호는 **그때 돌고 있는 Action**에만 간다. Action이 끝나면 그 Task들은 자동으로 반응을 멈춘다.

3. Context가 커지지 않는다

- 고정 정보(소켓 · VFX · 반경)는 Event의 필드, 매번 바뀌는 것(시전자 · 대상 · 토큰)만 Context.

5. 평가

잘 잡힌 것

1. 역할이 질문 하나씩

• 언제(BMC) · 무엇을(MPS) · 할 수 있나(SS) · 어떻게(Pattern). 각 부품이 다른 부품의 질문을 대신 답하지 않는다.

2. 공용 부분과 대상별 부분의 경계가 맞다

• SS · Runner · Action · Event는 플레이어도 그대로 쓴다. 바뀌는 건 "고르는 쪽"(MPS ↔ 입력 매핑)뿐.

3. 막는 것과 기다리는 것을 나눴다

• GAS에선 둘 다 AbilityTask. 여기선 Action(순서) / Task(조건)로 갈라 Task 수명 문제가 사라졌다.

4. 끝남을 이벤트로

• "끝났다"를 상태로 들고 있지 않아 BT 실행 순서와 무관. BT 용어(NodeState)가 공용 부품 밖으로 새지 않는다.

5. 재사용 기준이 서사와 일치

• "같은 코드, 다른 에셋" — 두 번째 보스는 MPS 슬롯 + Task 데이터만 바꾼다.

위험 · 아직 비어 있는 곳

1. SS가 슬롯을 받고 있다

• 지금 코드는 `GrantNormalPattern(NormalPatternSlot)` — 가중치 · 체력선(선택 정책)이 공용 부품 안으로 들어온다. SS는 **Pattern만** 받아야 한다.

2. 같은 값이 두 군데

• 쿨 = `Pattern._coolDown` + `NormalPatternSlot._cooldown`, 가중치 = `Pattern._weight` + `slot._weight`. 원천 하나로: 쿨 → Pattern, 가중치 → 슬롯.

3. 취소 경로가 없다

• 사망 · 기믹 끼어들기 때 Action.Cancel, 비동기 Event 정지(토큰), "정상 끝 vs 취소" 구분이 전부 미정.

4. 신호가 엉뚱한 Action에 갈 수 있다

• Action 사이 프레임(`_begun == false`)과 크로스페이드 중 이전 클립 이벤트.

5. 소환물 · VFX의 주인이 없다

• Event는 쏘고 잊는다. 패턴이 끊겼을 때 남은 소환몹 · VFX를 누가 회수하는지 정해야 한다.

6. 신호 입구가 아직 옛 길

• 프리팹의 Signal은 여전히 `BMC.FireEvent(PatternEvent)`를 직접 부른다 — 태그 경로를 우회.

구조 판단은 맞다. 남은 건 "경계를 코드가 지키게 하는 일"(1·2) + "끝나는 경로를 닫는 일"(3~5) + 입구 교체(6).

6. 지금 코드와 비교한 구현 목록

P0 이게 없으면 패턴이 안 돈다

1. MPS.TryPick 구현

• 기믹(체력선 넘고 미사용) 우선 → 아니면 일반 슬롯 중 `ss.CanUse(p)`만 모아 가중치 뽑기 · 직전 패턴 제외. `MonsterPatternSelector.cs`

2. 신호 수신기 → SS.SendTag

• `PatternSignal.cs`가 빈 껍데기. Animation Event(Object 인자 → GameplayTagSO) / Signal Receiver가 `SendTag(tag)`를 부르게 하고, 프리팹의 `FireEvent` 연결 교체.

3. PatternTask에 [Serializable]

• 없으면 Action의 `_tasks`가 인스펙터에 안 뜬다. `PatternTask.cs` (+ 불필요한 `using Unity.VisualScripting` 제거)

4. SS.BuildContext의 대상 null 체크

• `Managers.Party.GetCurrentCharacter().transform` — 파티가 없으면 NRE. `SkillSystem.cs`

5. 취소 토큰

• `cToken`이 비어 있어 Summon 같은 비동기 Event가 사망 후에도 돈다. SS가 패턴마다 CTS를 만들고 Cancel 때 끊기.

P1 돌기 시작하면 바로 드러날 것

1. 첫 사용 쿨 막힘

• 기록이 없으면 `lastUsedTime = 0` → 시작 후 쿨 길이만큼 못 씀. 기록 없음 = 사용 가능.

2. TryUseSkill 게이트 보강

• `IsBusy` · 부여 여부 검사. 지금은 `Runner.Start`가 먼저 Cancel해서 조용히 끊는다. 기믹용 끼어들기는 `interrupt` 옵션으로 따로.

3. OnEvent 가드

• `if (_pattern == null || !_begun) return;` · 안 쓰는 `ctx` 인자 제거. `PatternRunner.cs`

4. SS.Cancel + 사망 연결

• `HandleDeadState`의 `_runner.Cancel()`이 주석. SS.Cancel → Action.Cancel + CTS 취소. PatternEnded에 정상/취소 구분.

5. OnSpawn 초기화

• 쿨 기록 · 기믹 사용 표시 비우기, 등장 패턴(`_entrancePattern`) 시작. 풀 재사용 대비.

P2 경계 정리 · 잔재

1. SS는 Pattern만 Grant

- 슬롯 리스트 2개 → `HashSet<Pattern>` 하나. 쿨 → Pattern, 가중치 → 슬롯으로 중복 제거.

2. SS의 _state / NodeState 프로퍼티 삭제

- 이벤트 방식으로 바꾼 뒤 남은 잔재.

3. BMC 잔재 정리

- `_skills`(TimelineAsset 목록) · `FireEvent` · `BuildContext`(SS와 중복) · `CastAbility`(② 첫 시도 경로).

4. Director 위치 확인

- `TimelineAction`은 `owner.GetComponent<PlayableDirector>()` — BMC의 `_director`가 다른 오브젝트에 있으면 null.

5. 파일 인코딩 통일

- SkillSystem/ 7개 + SummonEvent가 CP949, 나머지는 UTF-8. 공개 저장소에서 한글 주석이 깨진다 → UTF-8로.

진행 순서 제안

1. P0 다섯 개 → 첫 보스 패턴 한 개를 끝까지

- TryPick · 신호 입구 · Serializable · null 체크 · 토큰. 망치 3연타 하나로 신호 → 반응까지 플레이 확인.

2. P1 — 끝나는 경로 닫기

- 쿨 첫 사용 · 게이트 · OnEvent 가드 · Cancel/사망 · OnSpawn. 기믹 끼어들기 확인.

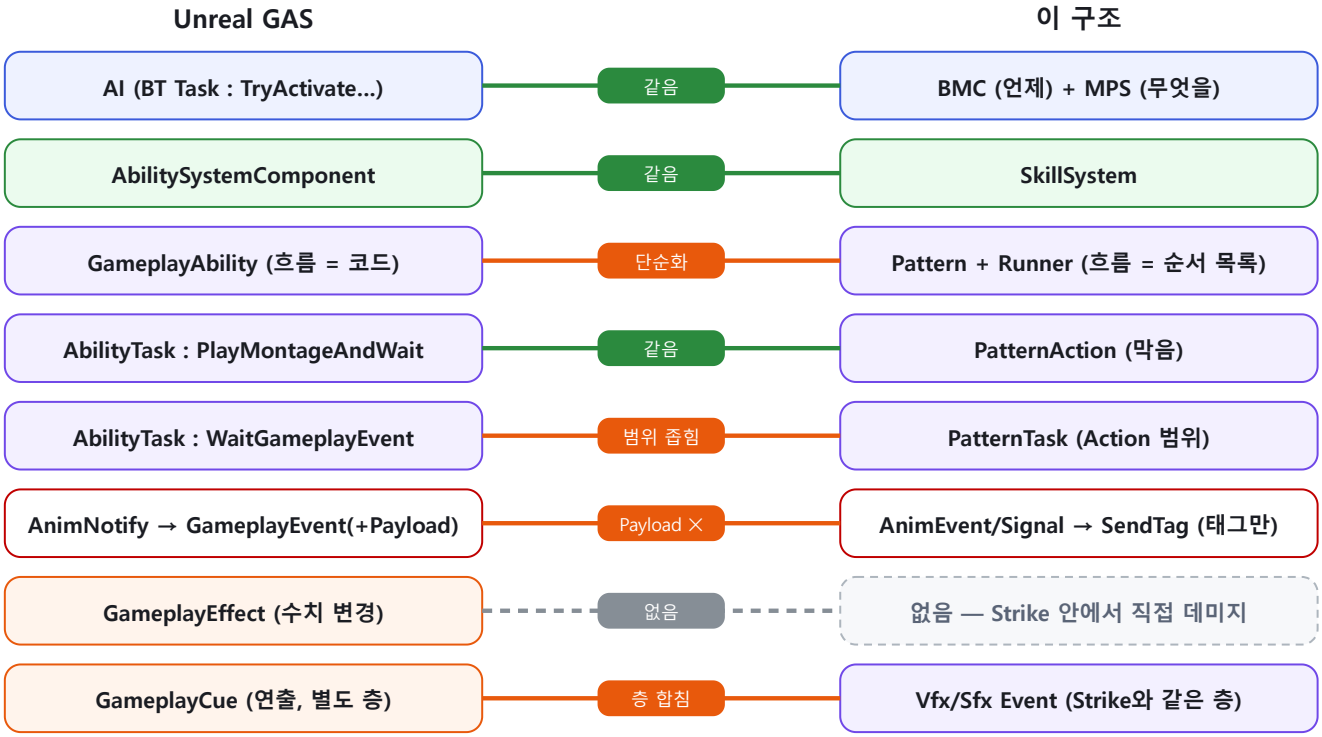
3. P2 — 경계 정리

- SS는 Pattern만, 값 중복 제거, 잔재 삭제, 인코딩.

4. 증명

- 두 번째 보스(텍스처 반전) = MPS 슬롯 + Task 데이터만 → 새 코드 0줄 확인.

7. GAS와 층별로 맞대 보기



위 4칸(선택 · 창구 · 흐름 · 동작)은 GAS와 같은 경계, 아래 4칸(신호 · 결과 · 연출)은 의도적으로 줄였다.

GAS	이 구조	관계	메모
GiveAbility	MPS.Awake → SS.Grant	같음	무엇을 줄지는 바깥 데이터가 정한다 — ASC는 목록만.
CanActivateAbility (Cooldown GE · Tag 요구)	CheckIsCoolDown · CheckCurrentTag	단순화	GAS 쿨 = 지속시간 GE가 거는 태그. 여기 = 마지막 사용 시각. 태그 검사는 아직 항상 true.
CommitAbility	Start 직후 쿨 기록	같음	"실행이 확정된 순간에 비용을 낸다" — 플레이어 Phase 1의 Commit과 같은 원칙.
ActivateAbility (그래프/코드)	List<PatternAction> 순차	단순화	분기 · 병렬이 없다. 보스 패턴엔 충분, 분기가 필요해지면 Action 하나로 감싼다.
WaitGameplayEvent	PatternTask	범위 좁힘	GAS Task는 어빌리티 전체에서 살아 있다. 여기선 그 Action 동안만 → 구독/해제 불필요.
GameplayEventData (Payload)	태그만	줄임	위치 · 대상은 Event가 Placement와 Context에서 직접 읽는다.
EndAbility / OnAbilityEnded	PatternEnded	같음	정상 / 취소 구분은 미구현
CancelAbility	—	미구현	사망 · 기믹 끼어들기에 필요 (P1-4).
Instancing Policy	프리팹 위 컴포넌트	고정	시전자마다 1개, 동시 1개 실행. 같은 패턴 중첩 실행은 불가(필요 없음).
Prediction · Replication	—	안 씀	완전 싱글(네트워크 코드 0).

8. GAS에서 가져오지 않은 것과 그 이유

지금 안 가져온 것

1. **GameplayEffect 층**
   • 데미지가 Strike 안에 있다. 지금은 보스만 쓰니 문제없지만, **플레이어 · 몹 · 보스가 같은 피격 규칙을 공유**하는 순간(서사의 증명 항목) 수치 변경을 한 곳으로 모아야 한다 → 다음 후보 1순위.
2. **GameplayCue 별도 층**
   • Vfx/Sfx를 Event로 같은 층에 둔 것은 "연출도 데이터로 붙인다"는 점에선 충분. 다만 **정리 책임**(끊겼을 때 VFX 회수) 때문에 소유자는 Pattern/SS로 둔다.
3. **Event Payload**
   • 신호에 데이터를 실으면 애니메이션이 "무엇을 할지"를 알게 된다. 신호는 사실만, 데이터는 Event 쪽 — 의도적으로 뺐다.
4. **Prediction · Replication · Attribute Set**
   • 싱글 게임, 스탯은 기존 MonsterStat이 이미 담당.

GAS를 두 번 — 이번엔 이름이 아니라 경계(선택 · 창구 · 흐름 · 동작)를 가져왔다.